

PATRONES DE DISEÑO

Los patrones de diseño varían en su complejidad, nivel de detalle y escala de aplicabilidad al sistema completo que se diseña. Me gusta la analogía de la construcción de carreteras: puedes hacer más segura una intersección instalando semáforos o construyendo un intercambiador completo de varios niveles con pasajes subterráneos para peatones.

Los desarrolladores pueden implementar estos patrones prácticamente en cualquier lenguaje. Al contrario que otros patrones, pueden utilizarse para diseñar la arquitectura de una aplicación completa.

Además, todos los patrones pueden clasificarse por su propósito. Este libro cubre tres grupos generales de patrones:

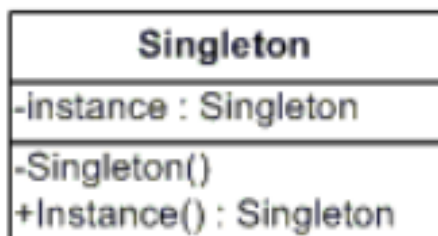
- Los patrones creacionales proporcionan mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización de código existente.
- Los patrones estructurales explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que se mantiene la flexibilidad y eficiencia de la estructura.
- Los patrones de comportamiento se encargan de una comunicación efectiva y la asignación de responsabilidades entre objetos.
- Los patrones de diseño son un juego de herramientas de soluciones comprobadas a problemas habituales en el diseño de software. Incluso aunque nunca te encuentres con estos problemas, conocer los patrones sigue siendo de utilidad, porque te enseña a resolver todo tipo de problemas utilizando principios del diseño orientado a objetos.
- Los patrones de diseño definen un lenguaje común que puedes utilizar con tus compañeros de equipo para comunicaros de forma más eficiente. Podrías decir: “Oh, utiliza un singleton para eso”, y todos entenderían la idea de tu sugerencia. No habría necesidad de explicar qué es un singleton si conocen el patrón y su nombre.

SINGLETON

Patrón creacional (se ocupan de cómo se crean los objetos)

El patrón de diseño Singleton garantiza que una clase tenga solo una instancia y proporciona un punto global de acceso a ella.

Una visualización de las clases y objetos que participan en este patrón.



Imagina que tenés una clase muy importante y **solo puede existir una única instancia** durante toda la ejecución del programa como la conexión de una base de datos, la

configuración de una aplicación, registros de logs, impresora compartida, etc. Si cualquier parte del programa pudiera hacer:

```
new Configuracion();
```

Tendría varias configuraciones distintas y eso traería inconsistencias, el Singleton evita eso.

Garantiza dos cosas, que solo exista un objeto de esa clase y que todos los demás utilizan exactamente ese mismo objeto.

Para evitar que alguien genere un objeto de esa clase escribiendo: **new Configuracion();** se hace que el constructor sea privado

```
private Configuracion()
```

```
{  
}
```

Al ser privado nadie de afuera puede crear objetos, la propia clase es la única que puede hacerlo.

Luego crea una instancia interna

(Cuando un atributo es static, pertenece a la clase y no a un objeto)

```
private static Configuración instancia = new Configuración();
```

Esta variable guarda el único objeto que existirá.

Después ofrece un acceso público

```
public static Configuración Instancia
```

```
{  
get {return instancia}  
}
```

Ahora cualquiera puede hacer:

```
Configuracion config = Configuracion.Instancia;
```

Pero nunca podrá hacer: `new configuracion`, porque el constructor es privado

Código completo:

```
public class Configuracion
```

```
{
```

```
    private static Configuracion instancia = new Configuracion();
```

```
    private Configuracion()
```

```
{  
}
```

```

public static Configuracion Instancia
{
    get { return instancia; }
}

public string Idioma { get; set; }
}

```

Ventajas

- Garantiza una única instancia.
- Evita duplicar recursos costosos.
- Centraliza información importante.
- Fácil de acceder desde cualquier parte del programa.

Desventajas:

No siempre es una buena idea usarlo.

Puede traer problemas porque:

- Acopla mucho el código.
- Hace más difíciles las pruebas (testing).
- Si muchas clases dependen del Singleton, luego cuesta modificar el sistema.
- Puede convertirse en una especie de "variable global", algo que suele evitarse en diseños modernos.

Por eso hoy se usa con más criterio que hace algunos años.

Pregunta	Respuesta
¿Qué resuelve?	Evita crear más de una instancia de una clase.
¿Cómo lo hace?	Constructor privado + una instancia estática + un método o propiedad pública para acceder a ella.
¿Cuántos objetos existen?	Solo uno.

¿Quién crea el objeto?

La propia clase.

¿Quién puede acceder?

Cualquier clase, mediante la propiedad o método estático.

La clase es `public` → cualquier otra clase puede verla y utilizarla.

El constructor es `private` → nadie puede hacer `new Configuracion()` desde afuera.

Clase pública → todos pueden usarla.

Constructor privado → nadie puede crear nuevas instancias.

Instancia estática pública → todos acceden al mismo objeto.

Porque su objetivo principal es controlar la creación de objetos. En lugar de permitir que se creen muchas instancias con `new`, garantiza que exista una sola y controla cómo se obtiene.

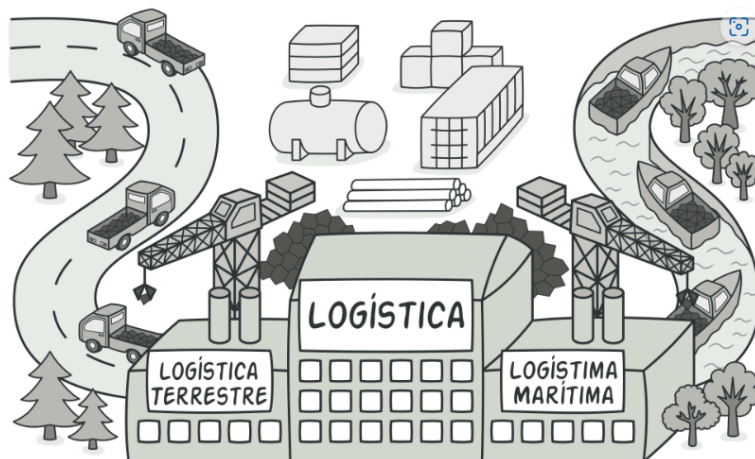
Analogía en el mundo real.

El gobierno es un ejemplo excelente del patrón Singleton. Un país sólo puede tener un gobierno oficial. Independientemente de las identidades personales de los individuos que forman el gobierno, el título “Gobierno de X” es un punto de acceso global que identifica al grupo de personas a cargo.

FACTORY METHOD

Patrones creacionales

Factory Method es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclasses alterar el tipo de objetos que se crearán.



No controla la cantidad de objetos sino cómo se crean

Funcionamiento: tiene 3 actores principales.

1- Producto: es la clase base: Vehículo

2- Productos concretos: Auto, Moto, Camión, todos son vehiculos

3- Factory: Tiene un metodo encargado de crear el objeto

En lugar de hacer: `new Auto();` hacemos:

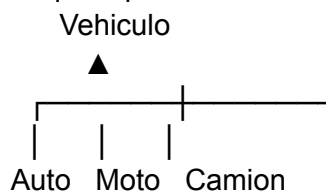
`Vehiculo v = fabrica.CrearVehiculo();`

Y la fábrica devuelve el objeto correspondiente.

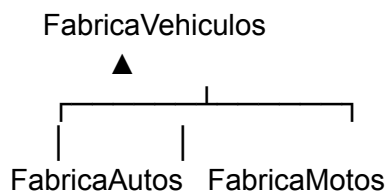
El patrón utiliza una **clase abstracta** llamada *Creator*, porque la idea es que las fábricas concretas **hereden** de ella y sobrescriban el método de creación.

Existen dos jerarquias distintas:

Jerarquías productos



Jerarquías de fábricas



Estas clases no son productos, sino que su trabajo es decidir que objeto crear.

Los constructores siguen estando en Auto, Moto, Camion, no se trasladan a la fábrica. La fábrica simplemente los invoca mediante `new`.

Creo que esa era la pieza que te faltaba: Factory Method no elimina ni reemplaza los constructores; solo evita que el cliente los llame directamente. El cliente le pide a la fábrica un objeto, y la fábrica decide qué constructor ejecutar.

Es decir:

- Sin Factory: el cliente decide qué crear.
- Con Factory: la fábrica decide qué crear.

¿Cuándo usar Factory Method?

El patrón Factory Method se utiliza cuando una aplicación necesita crear distintos tipos de objetos, pero el cliente no debe preocuparse por saber cuál crear. La decisión de qué objeto

instanciar se delega a una fábrica, lo que permite que el código sea más flexible y fácil de mantener.

Ejemplo: una empresa de transporte utiliza vehículos como Auto, Moto y Camión para realizar entregas. Si en el futuro incorpora un nuevo tipo de vehículo, como una Camioneta o un Dron, no será necesario modificar el código que solicita un vehículo; solo se actualizará la fábrica para que cree el nuevo tipo cuando corresponda.

Ventajas:

- Desacopla al cliente de las clases concretas que crea.
- Facilita agregar nuevos tipos de objetos sin modificar el código cliente.
- Centraliza la lógica de creación de objetos en un solo lugar.
- Favorece la reutilización y el mantenimiento del código.
- Cumple con el principio de abierto/cerrado (Open/Closed Principle): permite incorporar nuevas clases sin modificar las existentes.

Ejemplo en código:

1- Clase abstracta producto: clase padre

```
public abstract class Vehiculo
{
    public abstract void Conducir();
}
```

2- Productos concretos

```
public class Auto : Vehiculo
{
    public override void Conducir()
    {
        Console.WriteLine("Conduciendo un auto.");
    }
}

public class Moto : Vehiculo
{
    public override void Conducir()
    {
        Console.WriteLine("Conduciendo una moto.");
    }
}

public class Camion : Vehiculo
{
    public override void Conducir()
    {
        Console.WriteLine("Conduciendo un camión.");
    }
}
```

3- Clase abstracta Factory

```
public abstract class FabricaVehiculos
{
    public abstract Vehiculo CrearVehiculo();
}
```

```
}
```

Observá que esta clase no sabe qué vehículo crear. Solo obliga a que sus hijas implementen el método CrearVehiculo().

4- Fábricas concretas

Autos:

```
public class FabricaAutos : FabricaVehiculos
{
    public override Vehiculo CrearVehiculo()
    {
        return new Auto();
    }
}
```

Motos

```
public class FabricaMotos : FabricaVehiculos
{
    public override Vehiculo CrearVehiculo()
    {
        return new Moto();
    }
}
```

Camiones:

```
public class FabricaCamiones : FabricaVehiculos
{
    public override Vehiculo CrearVehiculo()
    {
        return new Camion();
    }
}
```

Programa principal:

```
FabricaVehiculos fabrica = new FabricaAutos();

Vehiculo vehiculo = fabrica.CrearVehiculo();

vehiculo.Conducir();
}
```

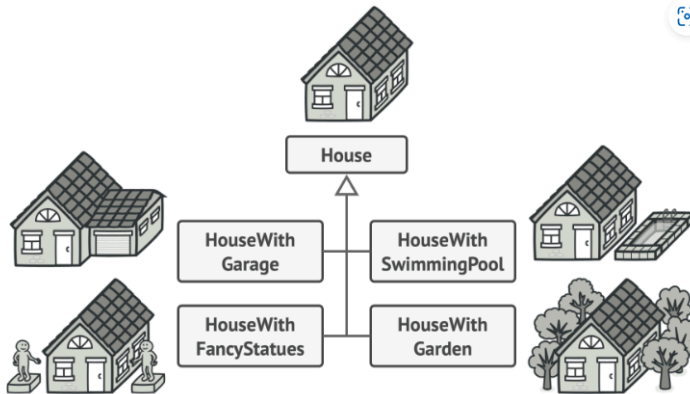
Salida:

Conduciendo un auto.

BUILDER

Creacional

Nos permite construir objetos complejos paso a paso. El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.



Este patrón responde a una pregunta muy distinta: **¿Cómo construir un objeto complejo paso a paso?**

En el ejemplo de las casas:

- Casa → es el producto.
 - Builder (ConstructorCasaLujo) → es el albañil o la cuadrilla que construye la casa.
 - Director → es el arquitecto o el jefe de obra que le dice al albañil qué hacer y en qué orden.
 - El arquitecto dice:
"Primero hacé las paredes."
 - El albañil las construye.
 - El arquitecto dice:
"Ahora hacé el techo."
 - El albañil lo construye.
 - El arquitecto dice:
"Ahora agregá el garage."
 - El albañil lo construye.
 - Al final, el albañil entrega la casa terminada.
-
- Casa = la vivienda terminada.
 - ConstructorCasaLujo = el equipo de albañiles especializado en casas de lujo.
 - ConstructorCasaSimple = el equipo que construye casas económicas.
 - Director = el arquitecto que coordina la obra.

Cada equipo sabe cómo construir una casa, pero el arquitecto puede decidir el orden de los pasos y asegurarse de que todas las obras sigan el mismo proceso.

Por eso se llama Builder ("constructor"): porque su responsabilidad es construir el producto paso a paso, igual que un albañil construye una casa ladrillo por ladrillo.

Permite construir un objeto complejo paso a paso, separando el proceso de construcción de la representación final del objeto. Su objetivo es evitar constructores con muchos parámetros y facilitar la creación de distintas versiones de un mismo objeto.

Elementos necesarios para implementar un builder

PRODUCTO:

Es el objeto complejo que se desea construir: contiene los atributos que se irán configurando durante el proceso de construcción.

Ejemplo: casa

con atributos:

- paredes
- techo
- garage
- pileta

BUILDER (CONSTRUCTOR)

Es una clase abstracta o interfaz que define los pasos necesarios para construir el producto. Tiene los métodos para construir la casa

Por ejemplo:

ConstruirParedes()

ConstruirTecho()

ConstruirGarage()

ObtenerCasa()

No construye la casa directamente, sino que define qué operaciones debe realizar cualquier constructor.

BUILDER CONCRETO

Es la clase que implementa los métodos definidos por el Builder.

Se encarga de construir el producto paso a paso.

Puede haber varios Builders concretos.

Por ejemplo:

ConstructorCasaSimple

ConstructorCasaLujo

Cada uno construye una casa distinta.

DIRECTOR (OPCIONAL)

Es la clase que coordina el proceso de construcción.

Indica el orden en que deben ejecutarse los pasos del Builder.

En implementaciones modernas puede omitirse, dejando que el cliente invoque directamente los métodos del Builder.

¿Cuándo se usa?

Builder se utiliza cuando un objeto:

- posee muchos atributos o configuraciones;
- debe construirse mediante varios pasos;
- puede tener diferentes representaciones sin modificar su estructura básica.

Ejemplo: empresa constructora desarrolla un sistema para construir distintos tipos de viviendas.

Todas las casas poseen la misma estructura básica, pero no todas incluyen las mismas características.

Por ejemplo:

- una Casa Simple posee paredes, techo y puertas;
- una Casa de Lujo además incorpora garage, pileta y jardín.

En lugar de crear múltiples constructores con diferentes combinaciones de parámetros, se utiliza el patrón Builder, donde cada constructor concreto se encarga de construir una versión específica de la casa paso a paso. De esta manera, el proceso de construcción es más organizado, reutilizable y fácil de extender si en el futuro aparecen nuevos tipos de viviendas.

Ventajas:

- Separa la construcción del objeto de su representación.
- Evita constructores con demasiados parámetros.
- Permite construir diferentes versiones del mismo objeto.
- Facilita el mantenimiento y la extensión del código.
- Hace el proceso de construcción más claro y reutilizable

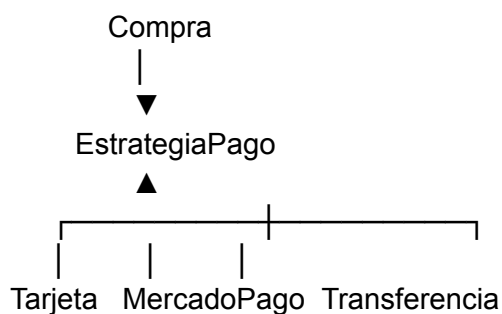
STRATEGY Comportamiento

Strategy es un patrón de diseño de comportamiento que te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.

El patrón Strategy se utiliza cuando un objeto puede realizar una misma tarea de distintas maneras, y queremos poder cambiar esa forma de ejecución sin modificar la clase principal. Permite intercambiar algoritmos o comportamientos en tiempo de ejecución.

Ejemplo real: en una aplicación de compras en cliente puede pagar con tarjeta de crédito, mercado pago, transferencia bancaria y el proceso siempre es el mismo: Realizar pago, lo único que cambia es como se realiza el pago.

Strategy separa el comportamiento en clases independientes.



ELEMENTOS QUE CONTIENE

1- LA CLASE PRINCIPAL: COMPRA con una referencia a una estrategia.

2- STRATEGY

Es una interfaz o clase abstracta y define el comportamiento comun: Pagar()

3- ESTRATEGIAS CONCRETAS

Implementación de la interfaz o herencia de clase abstracta

PagarTajeta, PagoTranferencia, PagoMercadoPago

Cuando usarlo

Cuando una clase puede realizar una tarea de diferentes maneras y querés poder cambiar ese comportamiento sin modificar la clase principal.

- Distintos medios de pago.
- Distintos algoritmos de ordenamiento.
- Distintas formas de enviar notificaciones (email, SMS, WhatsApp).
- Distintos métodos de compresión.
- Distintas rutas de navegación (auto, bicicleta, caminando).

Ventajas:

Evita grandes bloques de if o switch.

Permite agregar nuevos comportamientos sin modificar el código existente.

Cada estrategia tiene una única responsabilidad.

Facilita el mantenimiento y la reutilización del código.

Permite cambiar el comportamiento en tiempo de ejecución.

Strategy es un patrón de comportamiento que define una familia de algoritmos o comportamientos, los encapsula en clases independientes y permite intercambiarlos en tiempo de ejecución sin modificar la clase que los utiliza.

STATE Comportamiento

Perfecto. State es uno de los patrones que más se confunde con Strategy, porque ambos tienen una estructura muy parecida. Sin embargo, resuelven problemas completamente distintos.

Que resuelve?

El patrón State se utiliza cuando un objeto cambia su comportamiento según el estado en el que se encuentra. Es decir, el mismo objeto responde de manera diferente dependiendo de su estado actual.

Ejemplo:

Pensá en un semáforo.

Un semáforo puede estar en tres estados: Verde, Amarillo, Rojo

Y el comportamiento cambia según el estado.

Por ejemplo:

Si está en verde:

Cambiar → Amarillo

Si está en amarillo:

Cambiar → Rojo

Si está en rojo:

Cambiar → Verde

El objeto sigue siendo el mismo semáforo.

Lo único que cambia es su estado.

Otro ejemplo: Una puerta.

Puede estar:

- Abierta
- Cerrada
- Bloqueada

Si el usuario intenta abrirla:

- Si está cerrada → se abre.
- Si está abierta → no pasa nada.
- Si está bloqueada → muestra "Debe desbloquear la puerta".

La acción es la misma:

Abrir()

Pero el resultado cambia según el estado.

ELEMENTOS DE STATE:

1- CLASE PRINCIPAL: puerta

2- STATE: interfaz o clase abstracta

EstadoPuerta

define sus metodos comunes

Abrir()

Cerrar()

public interface IEstadoPuerta

```
{  
    void Abrir();  
}
```

3- ESTADOS CONCRETOS

Abierta

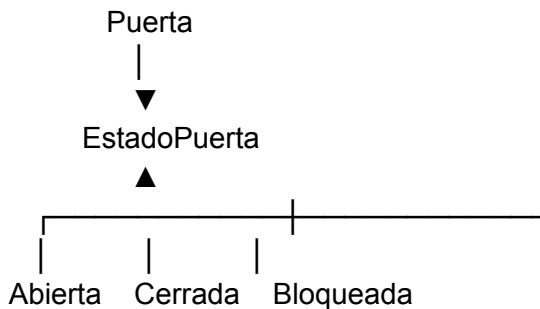
public class Abierta : IEstadoPuerta

```
{  
    public void Abrir()  
    {  
        Console.WriteLine("La puerta ya está abierta.");  
    }  
}
```

Cerrada

Bloqueada

Esquema:



Cuando usar STATE

Cuando un objeto:

- tiene varios estados;
- cambia de comportamiento según el estado actual;
- tendría muchos if o switch para controlar esos estados.

Ventajas:

- Elimina grandes bloques de if o switch.
- Cada estado tiene su propia responsabilidad.
- Facilita agregar nuevos estados.
- Hace el código más limpio y mantenible.

Strategy	State
Cambia el algoritmo .	Cambia el estado del objeto.
El cliente elige la estrategia.	El propio objeto cambia de estado.
Las estrategias suelen ser independientes.	Los estados normalmente conocen las transiciones entre ellos.

OBSERVER Comportamiento

Observer es un patrón de diseño de comportamiento que te permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.

¿Qué resuelve?

El patrón Observer se utiliza cuando un objeto cambia de estado y varios otros objetos deben enterarse automáticamente de ese cambio.

Permite que un objeto notifique automáticamente a otros objetos cuando ocurre un cambio.

Ejemplo vida real:

Pensá en un canal de YouTube.

Hay:

- Un creador de contenido.
- Miles de suscriptores.

Cuando el creador sube un video:

- YouTube les envía una notificación a todos los suscriptores.

El creador no llama uno por uno a cada persona.

Simplemente publica el video, y el sistema notifica automáticamente a todos los interesados.

Otro ejemplo:

Una estación meteorológica.

Cuando cambia la temperatura:

- se actualiza la pantalla;
- se registra en una base de datos;
- se envía una alerta al celular.

Todos reaccionan al mismo cambio.

Idea principal:

Existe un objeto llamado Sujeto (Subject).

Ese objeto mantiene una lista de observadores.

Cuando ocurre un cambio, recorre esa lista y avisa a todos.

ELEMENTOS PRINCIPALES

1- Subjet(sujeto): EL OBSERVADO

public class Sensor

```
{
    private List<IObservador> observadores = new List<IObservador>();

    public void AgregarObservador(IObservador observador)
    {
        observadores.Add(observador);
    }

    public void Notificar()
    {
        foreach (var observador in observadores)
        {
            observador.Actualizar();
        }
    }
}
```

2- OBSERVER

Interfaz o clase abstracta

Define un unico metodo: Actualizar()

```
public interface IObservador
{
    void Actualizar();
}
```

3- OBSERVADORES CONCRETOS

Implementan la interfaz:

Pantalla

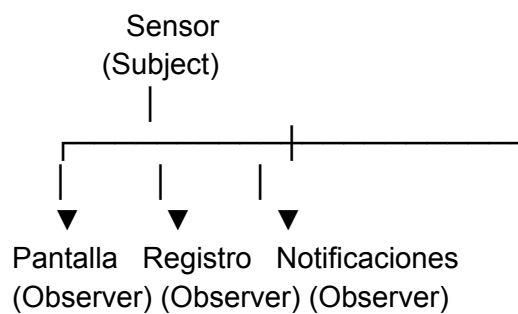
Registro

Notificaciones

```
public class Notificaciones : IObservador
```

```
{
    public void Actualizar()
    {
        Console.WriteLine("Notificación enviada.");
    }
}
```

Esquema:



Cuando usarlo: Cuando un cambio en un objeto debe reflejarse automáticamente en varios otros objetos.

Otros ejemplos reales:

Suscriptores de YouTube.

- Seguidores de Instagram cuando alguien publica.
- Sensores de temperatura.
- Bolsa de valores (cambia una acción y se actualizan varios paneles).
- Aplicaciones del clima.
- Interfaces gráficas (cuando cambia un dato, la pantalla se actualiza).

Ventajas:

Desacopla el sujeto de los observadores.

Permite agregar nuevos observadores sin modificar el sujeto.

Facilita la comunicación entre objetos.

Un mismo evento puede desencadenar múltiples acciones.

Strategy	State	Observer
Cambia un algoritmo.	Cambia el comportamiento según el estado.	Notifica cambios a otros objetos.

Un sistema de monitoreo de una fábrica. Un sensor detecta un aumento de temperatura. En ese momento deben ocurrir varias acciones al mismo tiempo: actualizar la pantalla de control, registrar el evento en la base de datos y enviar una alerta al supervisor. Con el patrón Observer, el sensor actúa como sujeto y los demás módulos como observadores. Cuando el sensor detecta el cambio, notifica automáticamente a todos los observadores, sin necesidad de conocer cómo funciona cada uno de ellos.

DECORATOR

Estructural

Decorator es un patrón de diseño estructural que te permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades.

¿Qué resuelve?

El patrón Decorator se utiliza cuando queremos agregar nuevas funcionalidades a un objeto sin modificar su clase original.

En otras palabras, permite añadir responsabilidades o características a un objeto de forma dinámica.

Pedís un café.

Podés elegir:

- Café solo.
- Café con leche.
- Café con crema.
- Café con chocolate.
- Café con leche y chocolate.
- Café con crema y chocolate.

Si hicieras una clase para cada combinación tendrías:

Cafe

CafeConLeche

CafeConCrema

CafeConChocolate

CafeConLecheYChocolate

CafeConCremaYChocolate

CafeConLecheCremaChocolate...
Serían muchísimas clases.
Decorator evita eso.

Idea principal:

En lugar de crear una clase para cada combinación, vamos agregando "decoradores" al objeto.

Es como envolver un regalo.

Primero tenés el regalo.

Después le agregás:

- papel,
- cinta,
- moño.

Cada envoltura agrega algo nuevo.

ELEMENTOS

1- COMPONENTE

Interfaz o clase abstracta comun.

Ibebida

```
public interface IBebida
{
    string Descripcion();
}
```

2-COMPONENTE CONCRETO

Es el objeto original

Café

```
public class Cafe : IBebida
{
    public string Descripcion()
    {
        return "Café";
    }
}
```

3- DECORADOR

Es una clase abstracta que también implementa IBebida y contiene una referencia a otra IBebida.

```
public abstract class Decorador : IBebida
{
    protected IBebida bebida;

    public Decorador(IBebida bebida)
    {
```

```

        this.bebida = bebida;
    }

    public abstract string Descripcion();
}

```

4- DECORADORES CONCRETOS

Leche, Chocolate, Crema

Cada uno agrega una característica

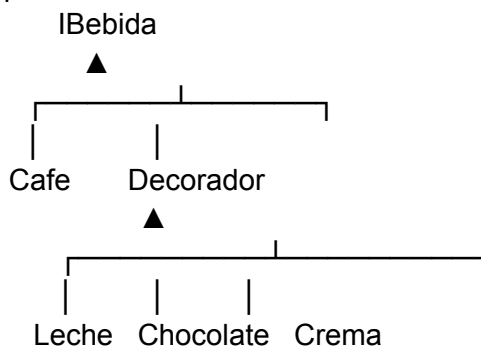
```

public class Leche : Decorador
{
    public Leche(IBebida bebida) : base(bebida)
    {
    }

    public override string Descripcion()
    {
        return bebida.Descripcion() + " con leche";
    }
}

```

Esquema:



Una cafetería permite personalizar las bebidas.

El cliente puede pedir un café y agregarle leche, chocolate, crema o cualquier combinación de estos ingredientes. En lugar de crear una clase para cada combinación posible, se utiliza el patrón Decorator, donde cada ingrediente es un decorador que envuelve al objeto original y agrega una nueva característica.

Ventaja:

Una cafetería permite personalizar las bebidas.

El cliente puede pedir un café y agregarle leche, chocolate, crema o cualquier combinación de estos ingredientes. En lugar de crear una clase para cada combinación posible, se utiliza el patrón Decorator, donde cada ingrediente es un decorador que envuelve al objeto original y agrega una nueva característica.

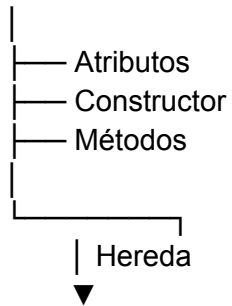
- Singleton → una única instancia.
- Factory Method → decide qué objeto crear.
- Builder → decide cómo construir un objeto complejo.
- Strategy → cambia el algoritmo o comportamiento.
- State → cambia el comportamiento según el estado.
- Observer → notifica cambios a varios objetos.
- Decorator → agrega nuevas funcionalidades a un objeto ya existente sin modificar su clase.

Patrón	Clasificación	¿Qué problema resuelve?	¿Cuándo se usa?	Idea principal	Ejemplo
Singleton	Creacional	Garantiza que exista una única instancia de una clase.	Cuando solo debe existir un objeto compartido en todo el sistema.	Una clase crea y administra su única instancia.	Configuración de una aplicación, conexión a una base de datos.
Factory Method	Creacional	Delega la creación de objetos a una fábrica.	Cuando existen varios tipos de objetos y no se quiere que el cliente conozca cuál crear.	La fábrica decide qué objeto concreto instanciar.	Empresa de transporte que crea Autos, Motos o Camiones.
Builder	Creacional	Construye objetos complejos paso a paso.	Cuando un objeto tiene muchos atributos o distintas formas de construirse.	Se separa el proceso de construcción del objeto final.	Construcción de una casa (simple o de lujo).

Strategy	Comportamiento	Permite cambiar un algoritmo o comportamiento sin modificar la clase principal.	Cuando una misma acción puede realizarse de diferentes maneras.	El comportamiento se encapsula en clases intercambiables.	Distintos métodos de pago (tarjeta, transferencia, Mercado Pago).
State	Comportamiento	Modifica el comportamiento de un objeto según su estado interno.	Cuando un objeto actúa diferente dependiendo del estado en el que se encuentra.	Cada estado implementa un comportamiento distinto.	Puerta abierta, cerrada o bloqueada; semáforo.
Observer	Comportamiento	Notifica automáticamente a varios objetos cuando otro cambia de estado.	Cuando varios objetos deben reaccionar a un mismo evento o cambio.	Un sujeto mantiene una lista de observadores y les avisa cuando ocurre un cambio.	Sensor que actualiza pantalla, registro y alarmas.
Decorator	Estructural	Agrega funcionalidades a un objeto sin modificar su clase.	Cuando se desean agregar características de forma dinámica.	Los decoradores envuelven al objeto original agregando nuevas funcionalidades.	Café con leche, chocolate y crema.

Herencia: estructura general

Clase Padre



Clase Hija

